KULLIYYAH OF INFORMATION & COMMUNICATION TECHNOLOGY
DEPARTMENT OF INFORMATION SYSTEMS

CSCI 2304 INTELLIGENT SYSTEMS
SEMESTER I, 2023/2024
SECTION 1

Group Project: Movie Recommender Using Cosine Similarity

PREPARED BY: Group F

NAME	MATRIC NUMBER	CONTRIBUTION
MOHAMMAD AFIQ IZ'AAN BIN MOHD ALI	2111977	100%
WAN AIMAN BIN WAN IBRAHIM	2113615	100%
NURIZUAN NAZRIN BIN KOMORI	2113021	100%

VIDEO DEMO LINK:
https://youtu.be/KEK4PL_Cd9E

LECTURER:
ASSOC. PROF. TS. DR. AMELIA RITAHANI BINTI ISMAIL

SUBMISSION DATE:
19 January 2023

TABLE OF CONTENTS

ABSTRACT **1**
1.0 INTRODUCTION **1**
1.1 BACKGROUND 2
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVES 3
1.4 MODEL OBJECTIVES 3
1.5 EXPECTED OUTPUT 4
2.0 LITERATURE REVIEW **4**
3.0 METHODOLOGY/EXPERIMENTAL SETUP **5**
3.1 DATASET 5
3.2 DATA LOADING AND MERGING 5
3.3 DATA CLEANING 6
3.4 DATA TRANSFORMATION 6
3.5 FEATURE EXTRACTION 7
3.6 TEXT PROCESSING 7
3.7 VECTORIZATION 7
3.8 SIMILARITY CALCULATION 8
3.9 RECOMMENDATION FUNCTION 8
3.10 MODEL PERSISTENCE 9
4.0 WEB APPLICATION **9**
5.0 RESULTS/DISCUSSION/IMPACT TO SOCIETY **12**
6.0 DEPLOYMENT **12**
7.0 ANALYSIS **12**
8.0 CONCLUSION **12**
9.0 REFERENCES **13**

ABSTRACT

This project leverages Python programming, utilizing Visual Studio Code, Anaconda, and Jupyter, for the building of a Movie Recommender System. The deployment is enabled by Streamlit, Pickle, and Requests packages. The primary recommendation algorithm utilized is content-based. The dataset, collected from The Movie Database, provides full movie facts such as id, title, cast, crew, and encompasses a huge array of genres. The main factor driving the recommendation process is the movie narrative summaries, harmonizing with the content-based algorithm. In overcoming barriers, getting a significant dataset and establishing an effective deployment plan were paramount. Extensive research and self-learning were vital in addressing these challenges. The outcomes of the project include the effective construction of a user-friendly website. The system allows users to input a movie title, providing tailored recommendations based on content analysis. User involvement is streamlined, requiring minimum input from users—simply providing a movie name triggers the algorithm to propose acceptable selections. The achieved results align well with the project's objectives, reflecting a successful and easy Movie Recommender System.

1.0 INTRODUCTION

In the field of data science and artificial intelligence, recommendation systems play a key role, giving tailored suggestions based on user preferences. This project looks into the building of a Movie Recommender System, employing Python within the Visual Studio Code, Anaconda, and Jupyter environment. The deployment is done through the integration of Streamlit, Pickle, and Requests packages, while the recommendation algorithm of choice is the content-based approach.

1.1 BACKGROUND

With the growth of movies across numerous genres, the standard methods of movie selection typically fall short in giving personalized suggestions. (Bhowmick, 2021). Traditional systems focus on broad ideas or individual preferences, resulting in a time-consuming and less customized movie-watching experience. To solve this, the project focuses on the development of an intelligent movie recommender system driven by a content-based algorithm. This system tries to examine individual user preferences and movie attributes, offering enhanced and tailored movie choices.

Navigating through huge movie databases remains inefficient, as people struggle to find movies fitting with their likes. The research overcomes this difficulty by creating a content-based algorithm that analyzes user preferences and tailors movie recommendations accordingly. The goal is to create an immersive movie choosing process, boosting the whole movie-watching experience.

1.2 PROBLEM STATEMENT

The wide range of movies accessible nowadays makes it hard for people to select ones that suit their likes. Traditional movie selection methods lack speed and customisation for a better viewing experience. Users are overwhelmed by the number of selections, making it difficult to find movies they enjoy. This project develops a content-based intelligent Movie Recommender System to solve this problem. The objective is to assess user preferences and movie attributes to provide personalized and relevant movie suggestions to improve movie-watching. The aim is to create a system that quickly and accurately searches massive movie collections and makes content-based recommendations.

1.3 OBJECTIVES

In creating a movie recommender system utilizing a content-based algorithm, the project objectives are outlined as follows:

1) **Implement Content-Based Algorithm for Recommendation:** Leverage content-based algorithms to boost the precision of movie suggestions based on particular user preferences.

2) **Swift User Preference Analysis:** Develop algorithms that swiftly and accurately analyze user preferences, employing the content of previously watched movies for timely and appropriate suggestions.

3) **Technological Integration:** Promote the integration of technology in the entertainment business by elevating the movie selection process through new content-based recommender algorithms.

1.4 MODEL OBJECTIVES

The precise aims for the content-based movie recommender system model include:

1) **Genre-Centric Recommendation:** Implement a content-based recommendation system that highlights movie genres, understanding and responding to users' individual genre preferences.

2) **Advanced Content Analysis:** Apply advanced content analysis algorithms to examine user behavior, history movie selections, and other relevant factors to boost the suggestion accuracy.

3) **Accurate Content-Driven Movie Recommendations:** Achieve the goal of generating a precise list of movie recommendations, curated based on the content qualities that fit with individual user interests.

1.5 EXPECTED OUTPUT

Upon successful implementation, the content-based movie recommender system will manifest the following outputs:

Movie title: Avengers		
Result 1: Avengers: Age of Ultron	**Result 2:** Iron Man	**Result 3:** Captain America: The First Avenger

*Assumption: The three results are related to the input title.

The system will leverage a library of movies and user preferences to employ content-based algorithms for recommendation purposes. When a user interacts with the system, providing input regarding their movie choices, the content-based algorithm will analyze these preferences, comparing them to the movie database. The outcome will be a curated list of movie suggestions based on content elements, offering viewers with a personalized and content-driven movie selection. Additionally, the system will record the date and time of the tip, ensuring that the suggestions remain relevant and up-to-date. At the completion of the engagement, consumers will witness a presentation of the recommended movies, illustrating the success of the content-based algorithm in adapting suggestions to individual preferences and boosting the entire movie-watching experience.

2.0 LITERATURE REVIEW

This literature review analyzes the impact of content-based filtering, focusing especially on the cosine similarity algorithm in movie recommender systems. The purpose of the review is to understand how through the use of cosine similarity techniques in content-based systems can generate personalized recommendations.

Within the field of movie recommender systems, researchers have examined several techniques that particularly include the cosine similarity technique, especially in content-based filtering situations (Yi, 2017).

Shubham Pawar et al. propose a content-based movie recommendation system using cosine similarity algorithm in which it matches user choices with movie genres and demographics (Pawar et al., 2022).

The hybrid model created by Rahul Katarya et al. combines the cosine similarity method using k-means clustering optimization to boost movie prediction accuracy. In combining both content-based and collaborative strategies, the study illustrates cosine similarity's adaptability in hybrid models (Katarya et al., 2017).

Although not focusing on cosine similarity, Hrisav Bhowmick et al. highlight the significance of content traits, such as genre, in influencing movie suggestions. Content-based features, which may be assessed using algorithms like cosine similarity, are important in genre-based recommendation systems (Bhowmick et al., 2020).

All in all, the literature highlights the importance of cosine similarity algorithms in creating tailored movie recommendations through content-based filtering. These methods will improve user satisfaction and engagement by concentrating on content qualities through the use of cosine similarity.

3.0 METHODOLOGY/EXPERIMENTAL SETUP

3.1 DATASET

The Movie Database (TMDb) was chosen as the dataset for the movie recommender system. We imported two datasets, 'tmdb_5000_movies.csv' and 'tmdb_5000_credits.csv' that contain significant data of 5000 movies for our project. For example, the movie title, genres, overview, cast, and crew will be needed for the recommender system.

3.2 DATA LOADING AND MERGING

The code loads the movie data from the CSV files using Pandas.

```
movies = pd.read_csv('data/tmdb_5000_movies.csv')
credits = pd.read_csv('data/tmdb_5000_credits.csv')
```

It merges the two data frames based on the movie title.

```
movies = movies.merge(credits, on = 'title')
```

Then, we only take relevant columns for modeling.

```
movies = movies[['movie_id', 'title', 'overview', 'genres', 'keywords', 'cast', 'crew',]]
```

3.3 DATA CLEANING

Null values are checked and dropped. Duplicate rows are also removed.

```
movies.isnull().sum()
```

```
movie_id    0
title       0
overview    3
genres      0
keywords    0
cast        0
crew        0
dtype: int64
```

```
movies.dropna(inplace=True)
```

```
movies.duplicated().sum()
```

```
0
```

3.4 DATA TRANSFORMATION

Columns containing JSON-formatted data (genres, keywords, cast, crew) are converted to lists using ast.literal_eval.

```
import ast
ast.literal_eval('[{"id": 28, "name": "Action"}, {"id": 12, "name": "Adventure"}, {"id": 14, "name'
```

```
[{'id': 28, 'name': 'Action'},
 {'id': 12, 'name': 'Adventure'},
 {'id': 14, 'name': 'Fantasy'},
 {'id': 878, 'name': 'Science Fiction'}]
```

```
import ast

def convert(text):
    l = []
    for i in ast.literal_eval(text):
        l.append(i['name'])
    return l
```

```
movies['genres'] = movies['genres'].apply(convert)
```

3.5 FEATURE EXTRACTION

Functions are defined to extract relevant information from the cast, crew, and overview columns, into a new column 'tags'.

```
movies['tags'] = movies['overview'] + movies['genres'] + movies['keywords'] + movies['cast'] + m
```

3.6 TEXT PROCESSING

Text data in the 'tags' column is preprocessed by removing spaces, converting to lowercase, and stemming using NLTK.

```
movies['overview'] = movies['overview'].apply(lambda x:x.split())
```

3.7 VECTORIZATION

Count Vectorization is applied to convert ‘tags’ into a numerical format suitable for modeling.

```
new_df['tags'] = new_df['tags'].apply(lambda x: " ".join(x))
new_df.head()
```

```
new_df['tags'] = new_df['tags'].apply(lambda x:x.lower())
```

3.8 SIMILARITY CALCULATION

Cosine similarity is calculated between movie tags, creating a similarity matrix.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(max_features=5000, stop_words='english')
```

```
vector = cv.fit_transform(new_df['tags']).toarray()
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
similarity = cosine_similarity(vector)
```

```
similarity
```

```
array([[1.        , 0.08346223, 0.0860309 , ..., 0.04499213, 0.        ,
        0.        ],
       [0.08346223, 1.        , 0.06063391, ..., 0.02378257, 0.        ,
        0.02615329],
       [0.0860309 , 0.06063391, 1.        , ..., 0.02451452, 0.        ,
        0.        ],
       ...,
       [0.04499213, 0.02378257, 0.02451452, ..., 1.        , 0.03962144,
        0.04229549],
       [0.        , 0.        , 0.        , ..., 0.03962144, 1.        ,
        0.08714204],
       [0.        , 0.02615329, 0.        , ..., 0.04229549, 0.08714204,
        1.        ]])
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
similarity = cosine_similarity(vector)
```

```
similarity
```

```
array([[1.        , 0.08346223, 0.0860309 , ..., 0.04499213, 0.        ,
        0.        ],
       [0.08346223, 1.        , 0.06063391, ..., 0.02378257, 0.        ,
        0.02615329],
       [0.0860309 , 0.06063391, 1.        , ..., 0.02451452, 0.        ,
        0.        ],
       ...,
       [0.04499213, 0.02378257, 0.02451452, ..., 1.        , 0.03962144,
        0.04229549],
       [0.        , 0.        , 0.        , ..., 0.03962144, 1.        ,
        0.08714204],
       [0.        , 0.02615329, 0.        , ..., 0.04229549, 0.08714204,
        1.        ]])
```

3.9 RECOMMENDATION FUNCTION

A function recommend is defined to provide movie recommendations based on similarity scores.

```
def recommend(movie):
    index = new_df[new_df['title'] == movie].index[0]
    distances = sorted(list(enumerate(similarity[index])), reverse = True, key = lambda x: x[1])
    for i in distances[1:6]:
        print(new_df.iloc[i[0]].title)
```

```
recommend('Spider-Man')
```

```
Spider-Man 3
Spider-Man 2
The Amazing Spider-Man 2
Arachnophobia
Kick-Ass
```

3.10 MODEL PERSISTENCE

The final movie dataframe and the similarity matrix are saved using Pickle for future use.

```
import pickle

pickle.dump(new_df, open('artifacts/movie_list.pkl', 'wb'))
pickle.dump(similarity, open('artifacts/similarity.pkl', 'wb'))
```

4.0 WEB APPLICATION

Importing Libraries:

```
import pickle
import streamlit as st
import requests
```

- **pickle**: Used for loading previously saved movie data and similarity matrix.
- **streamlit**: A Python library for creating web applications with simple Python scripts.
- **requests**: Used for making HTTP requests to fetch movie data from an external API.

‘fetch_poster’ Function:

```
def fetch_poster(movie_id):
    url = "https://api.themoviedb.org/3/movie/{}?api_key=3e2568e739a5e4f6cb7e30593068d0f2&language=en-US".format(movie_id)
    data = requests.get(url)
    data = data.json()
    poster_path = data['poster_path']
    full_path = "http://image.tmdb.org/t/p/w500/" + poster_path
    return full_path
```

- The function takes a ‘movie_id’ as an argument and constructs a URL to fetch movie details from TMDb.
- It retrieves the poster path from the API response and constructs the full path to the movie poster.

‘recommend’ Function:

```
def recommend(movie):
    index = movies[movies['title'] == movie].index[0]
    distances = sorted(list(enumerate(similarity[index])), reverse = True, key = lambda x: x[1])
    recommended_movies_name = []
    recommended_movies_poster = []
    for i in distances[1:6]:
        movie_id = movies.iloc[i[0]]['movie_id']
        recommended_movies_poster.append(fetch_poster(movie_id))
        recommended_movies_name.append(movies.iloc[i[0]].title)
    return recommended_movies_name, recommended_movies_poster
```

- The function takes a movie title (movie) as an argument and recommends similar movies based on content similarity.
- It uses the precomputed similarity matrix (similarity) and the loaded movie dataframe (movies) to find similar movies.
- The function returns lists of recommended movie names and their poster paths.

Streamlit Web Application:

```
st.header('Movie Recommendation System Using Cosine Similarity')
# Loads precomputed data (movies dataframe and similarity matrix)
movies = pickle.load(open('artifacts/movie_list.pkl', 'rb'))
similarity = pickle.load(open('artifacts/similarity.pkl', 'rb'))

# Creates a dropdown for selecting a movie
movie_list = movies['title'].values
selected_movie = st.selectbox(
    'Type or select a movie to get recommendation',
    movie_list
)
```

```
# Button to trigger the recommendation
if st.button('Show recommendation'):
    recommended_movies_name, recommended_movies_poster = recommend(selected_movie)
    # Displays recommended movies and their posters in five columns
    col1, col2, col3, col4, col5 = st.columns(5)
    with col1:
        st.text(recommended_movies_name[0])
        st.image(recommended_movies_poster[0])
```

- The Streamlit application is initiated with a header.
- It loads the pre-computed data (movies data frame and similarity matrix) using Pickle.
- A dropdown (select box) is created for selecting a movie from the dataset.
- A button is provided to trigger the recommendation process.
- When the button is clicked, the recommend function is called, and the recommended movies with their posters are displayed in a responsive layout using Streamlit columns.

5.0 RESULTS/DISCUSSION/IMPACT TO SOCIETY

From our tests, the movie recommender system ran successfully and are able to recommend a new or interrelated movie based on the movie title. The heart of this movie recommender system is the Cosine Similarity algorithm, which is able to measure similarities and correlation. (Mana & Sasipraba, 2021). Through the use of such an intelligent algorithm, the recommendations are not just any random selection of movies, but are intelligently curated to better match what the user wants.

In our opinion, even though the current techniques used are good enough for a movie recommender system, there is still room for improvements that can be implemented. As an overview, the current technique implemented is to search through similarities between two movie's summary, cast, genre and select few closest ones based on the vector optimization. To better improve our systems with the goal of personalized recommendation of movies, we could

implement user's previously watched movies, like and dislike, to create a personalization model, and include them as part of a feature in the clustering artificial intelligence algorithm.

Even though it is hard to find any relation between movie recommender system and Islamisation, at the core of this system, is a technology designed to analyze user preference and create recommendations. In a theoretical view, this core can be adapted as a content filtering system. As an example, the recommender system can be used as a form of parental control to recommend childrens content that contains Islamic values and way of life. One real life example of such a system is YouTube Kids, which limits its user to a few select videos. By taking the existing system as a baseline, and using our system as the foundation of the system, one can create an Islamic children-friendly content library that not only prioritizes age-appropriate entertainment but also aligns with Islamic values, fostering an enriching and culturally sensitive viewing experience for young audiences.

6.0 DEPLOYMENT

Currently, the movie recommender system was deployed in small scale, only as a prototype running locally on localhost. While the system has the potential to be published on the Internet, there were issues regarding the costly server and domain rental fees that kept it from happening.

Users can interact with the system through a web interface built with Streamlit, offering a more convenient and intuitive user experience compared to command-line interface. There were no authorization mechanisms in place, partly due to the static nature of the system which only operates in pre-trained dataset and no way of updating it. Another part is that the system are intended to be open and publicly used.

Data storage for this movie recommender system currently relied on Python pickle file, offering simple serialization of trained model and deserialization upon runtime. While this is a non-issue for the current prototype, it is hard to scale up as new data is added, which can sacrifice performance and storage. Thus, future revisions of this system should use a more scalable data storage solution such as SQL.

7.0 ANALYSIS

As part of evaluating the accuracy of an artificial intelligence, we would perform an analysis to have a look on whether the clustering artificial intelligence matches what a typical movie-avid person would expect from a recommender system. 5 movies are chosen, each with a reason for choosing, and the output recommendations will be tested with the criteria set.

Input title	Output recommendations	Feasible?
The Avengers [expected recommendations: Movies part of sequel]	Iron Man 3	Yes
	Avengers: Age of Ultron	Yes
	Captain America: Civil War	Yes
	Captain America: The First Avenger	Yes
	Iron Man	Yes
Pirates of the Caribbean: The Curse of the Black Pearl [expected recommendations: Movies of the same cast]	Pirates of the Caribbean: Dead Man's Chest	Yes
	Pirates of the Caribbean: On Stranger Tides	Yes
	Pirates of the Caribbean: At World's End	Yes
	VeggieTales: The Pirates Who Don't Do Anything	No
	The Pirates! In an Adventure with Scientists!	No
Inception [expected recommendations: Movies from same director]	12 Rounds	No
	RED	No
	Abduction	No
	Krrish	No
	The Animal	No
Tangled [expected recommendations: Animated movies]	Out of Inferno	No
	The Princess and The Frog	Yes

	Home on the Range	Yes
	Animals United	Yes
	Toy Story 3	Yes
Kung Fu Hustle [expected recommendations: Movies of same genre: Kung Fu]	Legend of a Rabbit	Yes
	Bulletproof Monk	No
	Kung Pow: Enter the Fist	Yes
	Kung Fu Panda 2	Yes
	Kung Fu Panda	Yes

From the analysis, we acquired a total of 16 true positives and 9 false positives. This results in an accuracy of 64% for our movie recommender system. Despite the barely-passing accuracy rate, we acknowledge the limitation of this method of analysis, which relies on human intuition for establishing expectations. This reliance on subjective criteria can lead to inconsistencies and a lack of robustness in the analysis, impacting the overall reliability of the system.

However, we stand on the objective of having artificial intelligence to match the user preference. Thus the key takeaway from this analysis is, we need to explore enhancements to the algorithm by incorporating more objective and criteria into the training process.

8.0 CONCLUSION

In summarizing the collaborative effort that our group has contributed towards the development of the movie recommender system, each member played a crucial role towards the success of the project. Aiman discovered the best dataset we could use for this assignment, Iz'aan coded the prototype of the system and Izuan performed analysis on the output. Together as a group we solved issues like data visualization, code debugging and deep understanding of the artificial algorithm.

Our testing phase confirmed the success of the movie recommender system, showcasing its ability to recommend new or interrelated movies based on a given title. The utilization of the Cosine Similarity algorithm at the core of this system demonstrated how intelligent systems can be achieved in a practical manner. While acknowledging the system's strengths, we also collectively recognized areas for improvement, particularly improving personalization by incorporating user feedback and user persona modeling.

The theoretical exploration of adapting the system for Islamic content filtering provided an intriguing avenue for potential future work. The core technology used in this movie recommender system could be repurposed for any Islamic-based systems such as Islamic children-friendly content recommender. This showcases the ability for our work to be adapted to future works, providing significant contributions towards society.

In the end, we learnt a lot about the use of artificial intelligence through this project. We have successfully addressed existing challenges, identified areas for improvement and opened doors to exciting possibilities for future work. This project not only represents our effort in learning about intelligent systems, but also serves as a foundation for ongoing exploration and enhancements in the field of intelligent content recommender systems.

9.0 REFERENCES

Lal, K. (2023, July 26). *"AJAX Movie Recommendation System with Sentiment Analysis" A content-based recommender system that recommends movies similar to the movie the user likes and analyses the sentiments of the reviews given by the user*. Commit 821f1ac, GitHub. Retrieved January 10, 2024, from https://github.com/kishan0725/AJAX-Movie-Recommendation-System-with-Sentiment-Analysis

Kharwal, A. (2020, May 20). *Movie recommendation system with machine learning*. Thecleverprogrammer. https://thecleverprogrammer.com/2020/05/20/data-science-project-movie-recommendation-system/

Srivani, K. D. (2022, August 28). *Movies recommendation system using Python*. Analytics Vidhya. https://www.analyticsvidhya.com/blog/2022/08/movies-recommendation-system-using-python/

Kumar, D. P., Singh, A. K., Arepu, S. N., Sarvasuddi, M., Gowtham, E., & Sanjana, Y. (2023). Content based recommendation system on movies. *Advances in Engineering Research*, 462-472. https://doi.org/10.2991/978-94-6463-252-1_49

Khatter, H., Goel, N., Gupta, N., & Gulati, M. (2021). Movie recommendation system using cosine similarity with sentiment analysis. *2021 Third International Conference on Inventive Research in Computing Applications (ICIRCA)*. https://doi.org/10.1109/icirca51532.2021.9544794

Yi, N., Li, C., Feng, X., & Shi, M. (2017). Design and implementation of movie recommender system based on graph database. 2017 14th Web Information Systems and Applications Conference (WISA). https://doi.org/10.1109/wisa.2017.34

Katarya, R., & Verma, O. P. (2017). An effective collaborative movie recommender system with cuckoo search. *Egyptian Informatics Journal*, 18(2), 105-112. https://doi.org/10.1016/j.eij.2016.10.002

Bhowmick, H., Chatterjee, A., & Sen, J. (2021). Comprehensive movie recommendation system. ResearchGate. [Preprint.] Retrieved January 12, 2024, from https://www.researchgate.net/publication/357301907_Comprehensive_Movie_Recommendation-System

Pawar, S., Patne, P., Ratanghayra, P., Dadhich, S., & Jaswal, S. (2022). Movies Recommendation System using Cosine Similarity. *International Journal of Innovative Science and Research Technology*, 7(4), 342-346. https://doi.org/10.5281/zenodo.6503164

Meteren, R.V., & Someren, M.V. (2000). Using Content-Based Filtering for Recommendation. Retrieved January 12, 2024, from https://users.ics.forth.gr/~potamias/mlnia/paper_6.pdf

Reddy, S., Nalluri, S., Kunisetti, S., Ashok, S., & Venkatesh, B. (2018). Content-based movie recommendation system using genre correlation. Smart Intelligent Computing and Applications, 391-397. https://doi.org/10.1007/978-981-13-1927-3_42

Pradeep, N., Mangalore, K. K. R., Rajpal, B., Prasad, N., & Shastri, R. (2020). Content based movie recommendation system. International Journal of Research in Industrial Engineering, 9(4), 337-348. https://doi.org/10.22105/riej.2020.259302.1156

Mana, S. C., & Sasipraba, T. (2021). Research on cosine similarity and Pearson correlation based recommendation models. Journal of Physics: Conference Series, 1770(1), 012014. https://doi.org/10.1088/1742-6596/1770/1/012014

Lü, L., Medo, M., Yeung, C. H., Zhang, Y., Zhang, Z., & Zhou, T. (2012). Recommender systems. Physics Reports, 519(1), 1–49. https://doi.org/10.1016/j.physrep.2012.02.006